



TD2 Java

Exercice n°1

1. Définir une classe `Personne` contenant deux attributs privés : `nom` et `prenom`. Munir cette classe d'un constructeur permettant l'initialisation de ses attributs et de la méthode `toString`
 2. Définir une classe `Etudiant` héritant de `Personne` et disposant d'un attribut privé `numInscription`, de type entier. Ajouter un constructeur qui permet de créer un étudiant à partir de son nom, son prénom et son numéro d'inscription et la méthode `toString()`
 3. Compléter la classe `Etudiants` qui gère un ensemble d'étudiants et changer les classes `Etudiant` et `Personne` en cas de besoin
- ```
class Etudiants
{
 // attributs
private Etudiant [] listeEtudiants;
private int nbEtudiants ;
//constructeur, reçoit en paramètre la capacité max du tableau listeEtudiants
Etudiants (int n){// ... }
// ajoute un étudiant dans le tableau s'il n'existe pas déjà et si le tableau //n'est pas plein
void ajouterEtudiant (Etudiant e) {//....}
// retourne l'étudiant ayant le numéro d'inscription passé en paramètre et
// null sinon
Etudiant rechercherEtudiant (int num) {//.... }
// affiche les informations sur tous le étudiants dans le tableau
void listerEtudiants (){//....}
}
```
4. Ecrire une classe `Test` contenant la méthode `main` et dans laquelle on testera les méthodes de la classe `Etudiants`

# Exercice n°2

Écrire les classes nécessaires au fonctionnement du programme suivant, en ne fournissant que les méthodes nécessaires à ce fonctionnement :

```
class TestMetiers {
 public static void main(String [] args) {
 Personne[] personnes = new Personne[4];
 personnes[0] = new Personne("Salah");
 personnes[1] = new Forgeron("Ali");
 personnes[2] = new Menuisier("Mohamed");
 personnes[3] = new Forgeron("Amor");
 for (int i=0 ; i<personnes.length ; i++)
 System.out.println (personnes[i]);
 }
}
```

Sortie de ce programme :

Je suis Salah

Je suis Ali le forgeron

Je suis Mohamed le menuisier

Je suis Amor le forgeron

# Exercice n°3

Qu’obtient- on sur écran à l’exécution du programme suivant ?

```
interface Son { void parle(); }
abstract class Mammifere implements Son
{ // Attributs
String nom, son;
int age;
Mammifere(String nom, String son, int age) // constructeur
{ this.nom = nom;
 this.son = son;
 this.age = age; }
public void parle() // Méthode parle affiche le son d'un mammifère
{ System.out.println(son+this);}
abstract void vitesse(); // Méthode abstraite vitesse
public String toString() // Méthode toString redéfinie
{ return(" mon nom est "+nom +" j'ai "+age+" ans");}
}
class Homme extends Mammifere
{ // Attributs
boolean marie;
Homme(String nom, String son, int age, boolean marie) // Constructeur
{ super(nom, son, age);
 this.marie = marie; }
void vitesse() // Implémentation de la méthode vitesse
{ System.out.println("Je cours à une vitesse de 20km/h");}
public String toString() // Méthode toString redéfinie
{ if (marie) return(super.toString()+" je suis marie");
Else return(super.toString()+ " je suis célibataire");
}
} // fin Homme
```

```
class Chien extends Mammifere
{ // Attributs
String race;
Chien(String nom, String son, int age, String race) // Constructeur
{ super(nom, son, age);
 this.race = race;}
void vitesse() // Implémentation de la méthode vitesse
{ System.out.println("Je cours à une vitesse de 30km/h");}
public String toString() // Méthode toString redéfinie
{ return(super.toString()+" j'appartient à la race des "+race);}
}
class Test{
public static void main(String [] args) {
Son []listeSons = new Son[2];
Mammifere []listeMammiferes = new Mammifere[2];
Homme h = new Homme("Ali", "Bonjour", 20, true);
Chien ch = new Chien("Snoopy", "Wouah!", 2,"caniches");
listeSons[0] = h;
listeSons[1] = ch;
listeMammiferes[0] = h;
listeMammiferes[1] = ch;
for (int i=0; i<listeSons.length; i++)
{ // la méthode parle est polymorphe
listeSons[i].parle();
// la méthode vitesse est polymorphe
listeMammiferes[i].vitesse(); }
} // fin main
} // fin Test
```

# Exercice n°4

La classe Robot modélise l'état et le comportement de robots virtuels. Chaque robot correspond à un objet qui est une instance de cette classe. Chaque robot :

- a un nom (attribut nom : chaîne de caractères)
- a une position : donnée par les attributs entiers x et y, sachant que x augmente en allant vers l'Est et y augmente en allant vers le Nord,
- a une direction : donnée par l'attribut direction qui prend une des valeurs "Nord", "Est", "Sud" ou "Ouest"
- peut avancer d'un pas en avant : avec la méthode sans paramètre avance()
- peut tourner à droite de 90° pour changer de direction (si sa direction était "Nord" elle devient "Est", si c'était "Est" elle devient "Sud", etc.) : avec la méthode sans paramètre droite(). Les robots ne peuvent pas tourner à gauche.
- peut afficher son état en détail (avec de simples System.out.println())

Le nom, la position et la direction d'un robot lui sont donnés au moment de sa création. Le nom est obligatoire mais on peut ne pas spécifier la position et la direction, qui sont définis par défaut à (0,0) et "Est".

## **Travail à faire:**

- 1) Écrire les instructions Java qui permettent de définir la classe Robot.
- 2) On veut améliorer ces robots en en créant une Nouvelle Génération, les RobotNG qui ne remplacent pas les anciens robots mais peuvent cohabiter avec eux. Les RobotNG savent faire la même chose mais aussi :
  - avancer de plusieurs pas en une seule fois grâce à une méthode avance() qui prend en paramètre le nombre de pas
  - tourner à gauche de 90° grâce à la méthode gauche()
  - faire demi-tour grâce à la méthode demiTour()Écrire cette nouvelle classe en spécialisant celle de la première question, sans modifier celle-ci : les nouvelles méthodes appellent les anciennes méthodes pour implémenter le nouveau comportement : avancer de n pas se fait en avançant de 1 pas n fois, « tourner à gauche » se fait en tournant 3 fois à droite, faire demi-tour se fait en tournant 2 fois